

One Person Lab

OPL Framework 白皮书

为长期运行的 AI 专业工作提供可组合、可验证、可演进的底座

优秀的智能体底座应让每项能力都能被独立理解、验证、替换和组合，同时让事实与责任始终清晰可见。

Cordis Host / Capability Domains / App Client Cordis / Governed Evolution

2026-08-15

Public whitepaper

目录

1 从“能运行”到“值得长期托付”	2
2 为什么选择 Cordis	2
3 四层生态中的可组合底座	2
4 四种身份，分别回答四个问题	3
5 受控组合，保持体验清楚	3
6 Framework 宿主与 App 客户端：同一架构，两种职责	4
7 两种界面实现，一份产品承诺	4
8 自进化：围绕模块持续改进	4
9 可组合的能力，清楚的责任	4
10 一次升级如何发生	5
11 为什么这套设计值得相信	5
12 结语	5

发布日期：2026-08-15

最近修订：2026-08-15

适用对象：希望理解 OPL Framework 为什么这样设计、它如何支撑可靠的 AI 专业工作，以及这套架构为什么能够长期演进的用户、合作者和技术决策者。

核心判断：优秀的智能体底座应让每项能力都能被独立理解、验证、替换和组合，同时让事实与责任始终清晰可见。

1 从“能运行”到“值得长期托付”

让一个智能体完成一次任务并不困难。真正困难的是让一组不断变化的模型、工具、专业能力和用户界面，在数周甚至数月的工作中仍然保持可理解、可恢复和可追责。

复杂知识工作会不断遇到变化：更好的模型出现了，新的资料源需要接入，执行方式需要替换，评估方法需要升级，界面也会迭代。如果这些能力都依赖同一套隐式启动顺序、全局状态和手工连接，任何一次升级都可能牵动整条运行链。系统越有能力，开发、测试和运维反而越困难。

OPL Framework 关注的是专业智能体如何持续变好：每一部分都能独立演进，整个系统仍然保持可信。

因此，Framework 把运行底座理解为一个可组合的能力系统。规则、发现、工作空间、阶段、执行、证据、呈现、评估和连接，都以清晰的能力域和责任面参与同一条专业工作线。它们彼此协作，也各自守护事实与判断。

2 为什么选择 Cordis

OPL Framework 选择 Cordis，是因为它提供了与长期目标一致的基本语法：能力通过明确的依赖进入组合，在清楚的生命周期中启动和退出，并能被检查、替换和重新组合。

这带来五个直接结果。

第一，**改变一个能力，不必先理解整个系统**。模块只需要理解自己的责任和明确依赖，升级执行器、目录或观测能力时，改动可以停留在合理边界内。

第二，**测试从整机验收变成模块证据**。每个模块可以独立装载、注入受控环境、面对缺失依赖并证明退出后没有遗留影响；完整组合再验证模块之间是否协作正确。

第三，**故障可以被解释**。系统能够回答实际装载了什么、哪项能力由谁提供、哪个版本参与了本次工作，以及不兼容发生在哪里。

第四，**升级和回退有了稳定对象**。一次工作可以绑定到明确的组合快照。新模块可以先在受控组合中验证，出现问题时也能回到上一组已知配置，而不必依赖难以复现的全局状态。

第五，**自进化有了真实的操作单位**。候选生成、独立评估、受控替换和效果比较可以围绕一个模块进行，不必每次修改整个智能体运行时。

Cordis 提供组合秩序，专业智能体与负责人提供领域判断。运行层记录能力如何协作，领域层决定成果是否成立；这种分工让两种专业性各得其所。

3 四层生态中的可组合底座

用户面对的是 OPL Base、OPL App、OPL Packages 和正在落地的 OPL Cloud。Framework 是 Base 的核心实现和唯一 Cordis Host；Packages 提供可独立安装与发布的专业能力；App 提供本地工作台；Cloud 提供在线治理、托管资源与协作服务。Cordis 在 Base 内部组织能力，让四层产品使用同一套清楚的工作语言。

OPL 不再用历史“十大模块”解释不断变化的实现，也不把 13 个源码单元或 Cordis plugin 名称直接暴露给用户。家族品牌组合只保留真实、可辨认的产品责任，并按一项工作从准备到运营的四段旅程展示。当前组合有 11 个品牌，数量不是承诺；真实责任发生合并、拆分或新增时，品牌组合随之调整。

旅程	品牌	对用户的承诺	命名判断
Foundation	OPL Charter	定规则、守边界	保留：短、独特，准确表达宪章
Foundation	OPL Workspace	让每项工作有正确位置	保留：虽通用，但最容易理解且已有跨端认知
Build	OPL Atlas	找到所有可用能力	保留：发现隐喻鲜明，与 Connect 分工不同
Build	OPL Pack	把能力变成可描述、可安装单元	保留：简短，直接对应 Package 与 ABI
Build	OPL Stagecraft	设计专业工作的阶段与上下文	保留：辨识度高，又不冒充领域判断
Run	OPL Runway	让工作启动、持续、恢复和收口	保留：执行隐喻清楚
Run	OPL Ledger	留下可追溯的证据与事件	保留：天然表达可核验记录
Run	OPL Connect	接通外部来源、carrier 与生态	保留：动作性强，不与 Atlas 合并
Operate	OPL Console	看清状态并采取行动	保留：已有 App、Cloud 与 Framework 的真实分层 surface
Operate	OPL Foundry	用证据锻造更好的 Agent	简化：品牌去掉 Kernel，Kernel 只表示内部实现角色
Operate	OPL Fabric	供给并治理托管资源	保留：已有独立 Cloud 资源 authority 与生命周期

这不是两套真相。家族 capability registry 是唯一品牌目录；Framework 的十项 registry 只是其中具有 CLI、L4 或 L5 surface 的投影，所以 Cloud-owned Fabric 不会伪装成 Framework module。每个品牌都指向当前真实 authority、source topology、Package 和 Cordis contribution；默认随责任边界同步调整，只有跨产品或独立生命周期的现实边界才保持非一对一关系。

4 四种身份，分别回答四个问题

为了让管理、开发和发布使用同一份地图，OPL 把同一能力的四种身份分别记录：

身份	回答的问题
品牌 / 能力域	读者如何理解一项稳定的产品责任？
责任方	谁对事实、质量、权限、状态或发布作出决定？
OPL Package	哪些能力可以独立安装、发布、升级和分发？
Cordis 能力单元	哪段运行能力需要独立挂载、注入、观察和退出？

因此，一个 Package 可以承载一组相互关联的能力，一个品牌也可以在不同产品面提供贡献；组合快照则为每次运行记录实际采用的能力和来源，让版本、责任与回退对象保持清楚。

独立 Package、版本线和仓库会优先服务于具有独立发布节奏与替换价值的 Runway Executor、Foundry Evaluation、Connect Discovery、Package Host 与 Cordis ABI 等单元。Framework 先以统一的源代码和 组合快照建立清楚边界，再根据真实发布证据推进独立分发，让模块化带来实际收益。

5 受控组合，保持体验清楚

OPL Framework 在内部保留能力组合的自由，在产品中提供少量经过完整验证的组合形态。

基础运行、完整桌面工作台、研究工作、基金工作、视觉交付和 Foundry 开发，需要的能力组合各有侧重。Framework 用受控配置表达这些差异：每种配置都选择明确的能力，冻结实际组合，并接受完整验证。

用户只需选择“我要完成什么工作”。稳定的能力域降低维护成本，受控配置降低使用成本；二者结合，让系统内部快速演进，产品体验仍然稳定、清楚、可预测。

6 Framework 宿主与 App 客户端：同一架构，两种职责

OPL Framework 是唯一的 Cordis 宿主，负责选择进入运行组合的能力、管理生命周期，并向 App 提供经过产品准入的能力和状态。

One Person Lab App 以客户端 Cordis 组织窗口、导航、任务视图、文件工作面、进度呈现和操作入口。它使用 Framework 提供的能力，并由 App 统一决定这些能力如何成为连贯的用户体验。

宿主与客户端使用一致的插件思想并承担互补责任：宿主保证能力真实存在、可以被可靠使用；客户端把这些能力组织成稳定的页面、动作和交互。Framework 负责运行组合，App 负责产品准入、页面结构和发布。

这种分工让运行底座与界面都能独立演进，同时共享同一套产品事实。

7 两种界面实现，一份产品承诺

One Person Lab App 当前同时推进两种 renderer/carrier：基于 AionUI 的主线版本，以及 DSH-derived Studio 候选。它们是同一个 App、同一个 Host-derived Client Cordis 架构的不同实现，不是两套产品或两个控制面。

两条路线遵循同一顶层设计：Framework 提供 Host-projected、allowlisted client graph，App 客户端 Cordis 组织界面能力，产品合同定义用户应当看见的任务、能力包、状态、动作和责任边界。AionUI 与 Studio 必须共享同一 product profile、typed slots/actions、RPC/events 和 state semantics；Client 不能自行发现或安装 plugin，也不能维护第二份 Package currentness、action 或 release truth。

AionUI 已通过 App compatibility admission，继续承载当前 active 产品体验；Studio 也已在 canonical candidate caller 上完成同一 profile、Client Cordis、typed slot/action、state readback 和跨 GUI conformance，但没有替换 active shell。未来切换仍是 App owner 的显式 adapter selection 与重新准入，不是未经验证的任意热切换，也不因为候选验证通过而自动获得 release-ready 身份。

8 自进化：围绕模块持续改进

Harness 的长期价值，在于让系统能够观察自身、提出改进并在证据约束下采用更好的实现。

当执行、发现、观测、评估和连接都成为独立能力单元或 Package 时，Foundry 可以围绕一个清楚对象工作：冻结当前版本，生成候选，执行针对性测试，在真实组合中比较效果，再由明确的责任方决定是否激活。成熟的改进只替换相应模块，其他能力保持稳定。

模块身份提供实验单位，组合快照提供复现边界，Ledger 提供证据，Foundry Evaluation 提供独立比较，最终采用决定归属明确。自进化由此获得可以验证、可以回退、可以持续积累的工程基础。

自进化因此成为一条专业的改进链：提出候选、获得证据、接受评估、受控激活、保留回退。

9 可组合的能力，清楚的责任

可组合层负责运行能力之间的协作，各类事实则由真正的拥有者维护。

领域智能体决定专业成果是否合格；Workspace 和文件系统保存实际材料与产物；持久运行环境保存长期执行历史；Ledger 保存证据与事件；App 持有桌面产品和发布事实；能力包负责人维护来源、版本与能力声明。Framework 把这些事实连接起来，让每项判断都能回到正确的责任方。

这种分工让新的提供者、连接器、观察器或执行器能够进入组合，同时保持事实归属稳定。创新可以发生在局部，信任则贯穿整个系统。

10 一次升级如何发生

假设 Runway Executor 出现了一个更擅长长期任务恢复的候选版本。

在传统整体式运行器中，这次升级可能同时触碰启动逻辑、状态页、工具注册、任务恢复和发布脚本。团队需要重新验证整个系统，而且很难判断改进究竟来自哪里。

在 OPL Framework 中，候选版本拥有自己的模块身份。它先在受控 Profile 中替换现有 Runway Executor，沿用相同的 Stagecraft 上下文、Workspace 位置和 Ledger 证据边界。Foundry Evaluation 比较恢复成功率、证据完整性和用户可见结果。只有证据支持且 owner 接受后，新组合才成为后续工作的选择。

升级前后的工作都能回答：使用了哪个模块版本，组合中还有哪些能力，评估依据是什么，出现问题时回到哪里。一次局部改进因此不会变成一次全系统赌博。

11 为什么这套设计值得相信

OPL Framework 的专业性不来自模块数量，而来自一组持续兑现的工程承诺。

- **概念与实现一致。** 唯一品牌组合把 capability domain 与真实 authority、源码、能力包和 Cordis contribution 显式关联，而不伪造一对一关系。
- **组合可以被看见。** 每次运行都能解释实际装载的能力及其来源。
- **升级可以被比较。** 候选模块先获得独立证据，再进入受控组合。
- **故障可以被隔离。** 模块生命周期和责任边界让问题停留在合理范围，并保留清楚的回退对象。
- **界面演进保持产品连续。** AionUI 主线与 DSH-derived Studio 候选共享同一能力投影、App 产品合同和准入门。
- **责任始终归属明确。** Framework 负责组合与运行，领域质量、文件、发布和最终决定归对应责任方。

这套设计让 OPL 可以同时获得两种通常难以兼得的能力：内部持续变化，外部长期稳定。

12 结语

OPL Framework 的目标，是成为一套结构清楚、持续进化的专业能力系统。

四层生态让用户理解 Base、App、Packages 和 Cloud 的分工；清楚的认知域、责任方、能力包与插件贡献让开发者能准确定位边界；Cordis 让运行贡献可以组合、检查和替换；受控 Profile 让内部自由不会成为用户负担；Framework 宿主与 App 客户端 Cordis 让运行和界面共享同一种设计语言；Foundry 则让每项能力可以在证据约束下持续改进。

最终，用户不需要感知插件系统本身。他们感知到的是更稳定的工作、更清楚的进展、更容易恢复的任务，以及一个能够不断升级却不丢失责任与事实的 AI 专业平台。

了解更多：

- [One Person Lab 家族白皮书](#)
- [One Person Lab](#)
- [One Person Lab App](#)
- [OPL Cloud](#)